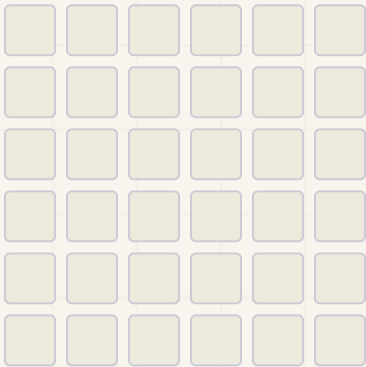


Aula 3

Modelos **Encoder** — o caso BERT

Do bloco transformer genérico a um modelo completo: como empilhar camadas, pré-treinar sem rótulos, e depois especializar (fine-tuning) para tarefas concretas de NLP.



Da camada ao modelo encoder

Já conhecemos a camada transformer inteira (eq. 12.13). Hoje empilhamos K delas e olhamos para o primeiro dos três "tipos" de modelo: o encoder.

12.5.3	Três tipos de modelo	encoder · decoder · enc-dec	12.6.1	Pré-treinamento	masked language modeling
→	Interação encoder	todos com todos	12.6.2	Fine-tuning	classificação, NER
12.6	BERT	arquitetura & escala	→	Fechamento	ponte para a Aula 4

Três formas de usar as mesmas camadas

Tokenização e embeddings (aula 1) produzem a matriz X . Ela passa por uma série de K camadas transformer — chamada de *transformer model*. O que muda entre os três tipos é como as camadas atendem umas às outras e o que se faz com a saída.

ENCODER

Transforma o texto numa representação rica, reaproveitável em várias tarefas. Todo token atende a todo token.

DECODER

Prevê o próximo token da sequência. Cada token só atende a si mesmo e aos anteriores. Assunto da Aula 4.

ENCODER-DECODER

Combina os dois para tarefas sequência→sequência, como tradução. Também na Aula 4.

Hoje: encoder, usando BERT como exemplo concreto (seção 12.6).

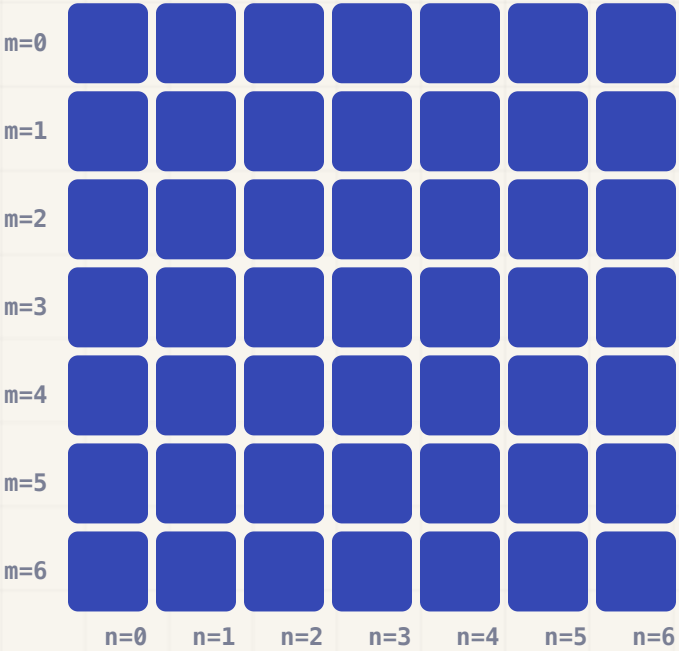
Empilhando K camadas

Lembre-se: cada camada preserva a forma $D \times N$ da entrada — é exatamente isso que permite empilhar quantas quisermos, uma alimentando a próxima.



Todo token conversa com todo token

Num encoder, a self-attention é aplicada sem nenhuma restrição: cada posição pode atender a qualquer outra posição da sequência — inclusive as que vêm depois dela (contexto bidirecional).



linha = posição que consulta (query) · coluna = posição consultada (key) · célula azul = atenção permitida

BERT em números

BERT é um encoder concreto — um exemplo de tamanho real das quantidades que vimos nas aulas anteriores.

VOCABULÁRIO & EMBEDDINGS

30.000 tokens no vocabulário, cada um convertido num embedding de dimensão $D = 1024$.

CAMADAS & HEADS

24 camadas transformer, cada uma com self-attention de 16 heads.

DIMENSÕES INTERNAS

$\Omega_{vh}, \Omega_{qh}, \Omega_{kh}$ são 64×1024 ($D/H = 1024/16 = 64$).
MLP com camada oculta de 4096.

Total: ~340 milhões de parâmetros — enorme quando o BERT foi lançado (2018), pequeno perto dos modelos de hoje.

Aprender sem rótulos: prever a palavra escondida

A auto-supervisão do BERT: substituir aleatoriamente algumas palavras por um token genérico **<mask>** e pedir ao modelo que recupere a palavra original a partir do embedding de saída daquela posição.

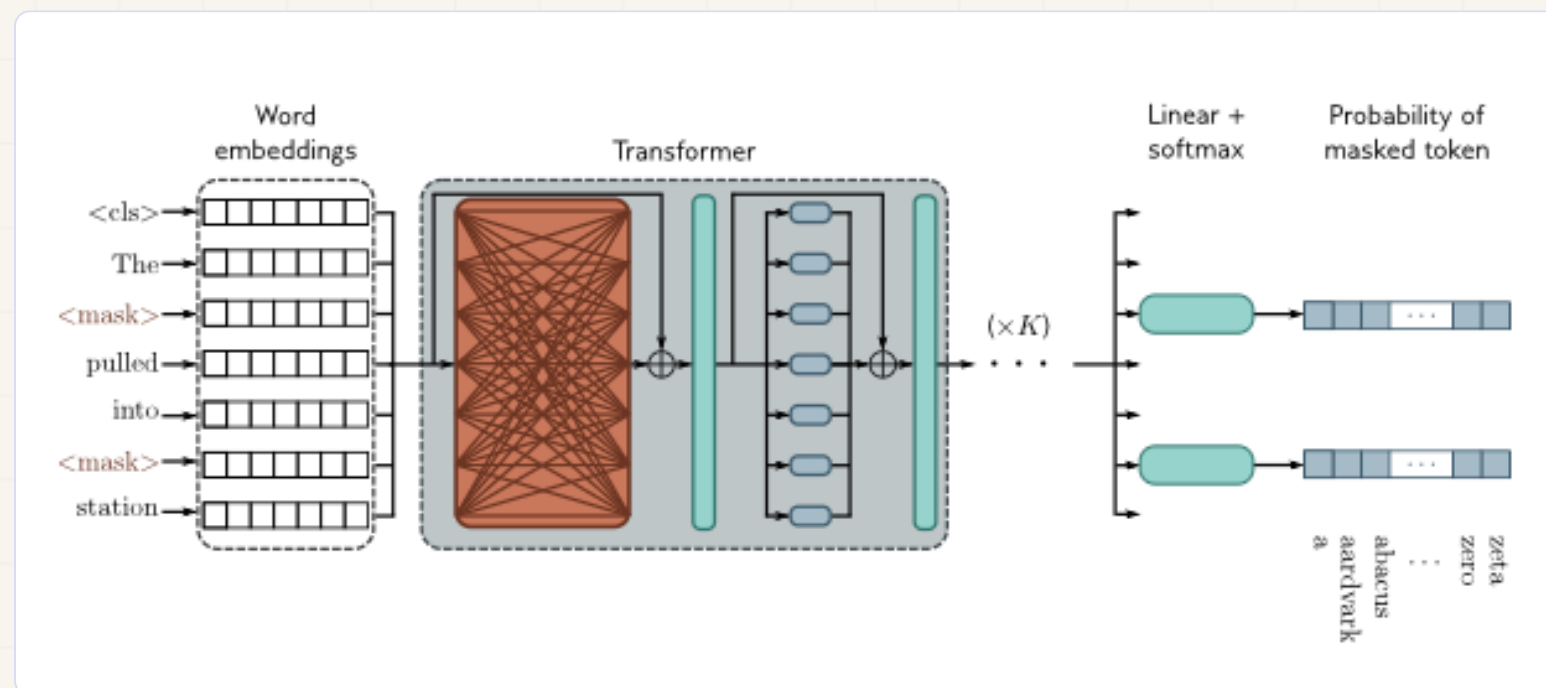


Figura 12.10 – Prince, *Understanding Deep Learning* (2023)

O embedding de saída na posição do **<mask>** passa por um softmax sobre o vocabulário inteiro — perda de classificação multiclasse.

A escala do pré-treinamento

A auto-supervisão permite usar quantidades enormes de texto não rotulado — o "custo" é que cada token processado só contribui para a perda ocasionalmente.

DADOS & SEQUÊNCIA

Máx. 512 tokens por entrada · corpus de 3.3 bilhões de palavras.

TREINAMENTO

Batch de 256 · 1 milhão de passos · $\approx$ 50 épocas.

TAREFA SECUNDÁRIA

Classificar se duas frases eram adjacentes no texto original — ganho apenas marginal.

Ineficiência de dados: são necessários 7 tokens processados para gerar apenas 2 termos na função de perda (só as posições mascaradas contam).

O que prever palavras ausentes ensina

Para prever bem a palavra escondida, o modelo é forçado a absorver regularidades de sintaxe e algum "senso comum" superficial sobre o mundo.

O *<mask>* entrou na estação.

P(trem | contexto) >> P(amendoim | contexto)

Também aprende regularidades sintáticas — por exemplo, que o adjetivo *vermelho* aparece com frequência antes de substantivos como *casa* ou *carro*, mas nunca antes de um verbo como *gritar*. Ainda assim, o quanto esse tipo de modelo realmente "entende" o mundo é limitado.

Especializar um modelo genérico

Depois do pré-treinamento, os mesmos pesos são reaproveitados (transfer learning): acrescenta-se uma camada extra — linear ou um pequeno MLP — para converter os embeddings de saída no formato da tarefa desejada, e ajusta-se tudo com um conjunto de dados rotulado, bem menor.



Classificação de texto: o token <cls>

Um token especial <cls> é colocado no início de cada sequência (já durante o pré-treinamento). Para classificar a sequência inteira, usa-se só o embedding de saída dessa posição.

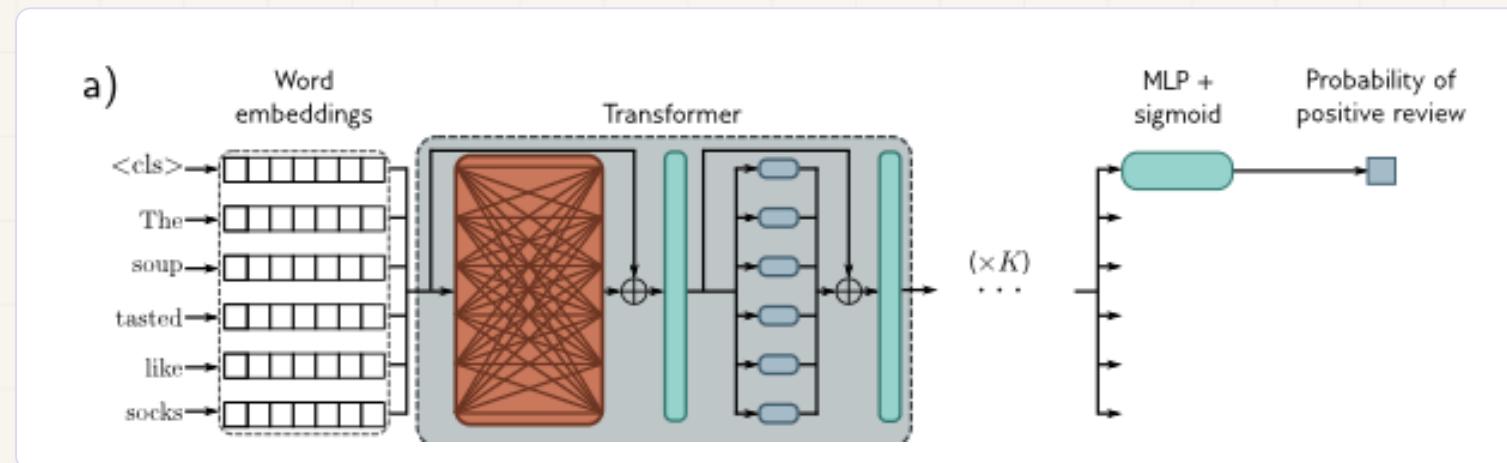


Figura 12.11a – Prince, *Understanding Deep Learning* (2023)

Exemplo — análise de sentimento: o vetor de <cls> é mapeado para um único número e passado por uma sigmoid, contribuindo para uma perda de entropia cruzada binária (positivo / negativo).

Classificação por palavra: NER

No reconhecimento de entidades nomeadas, cada embedding x_n (não só o <cls>) é mapeado para um vetor $E \times 1$, um valor por tipo de entidade possível.

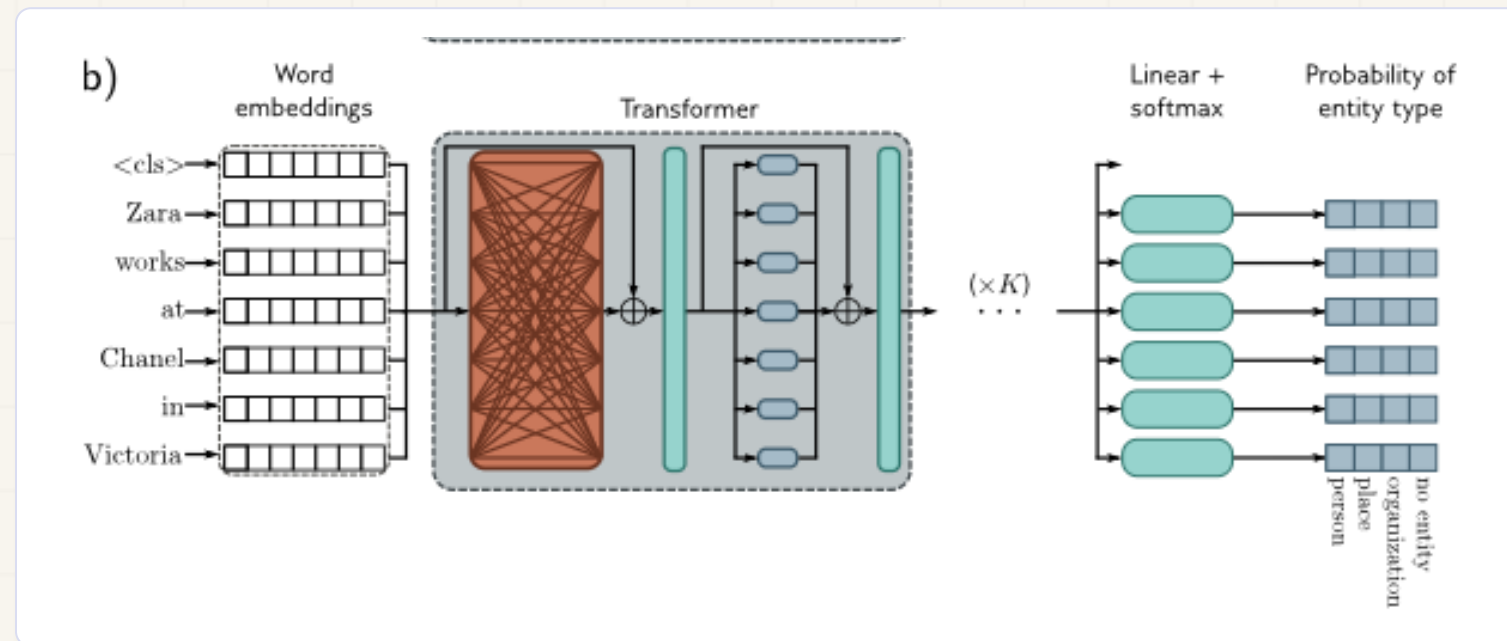


Figura 12.11b – Prince, *Understanding Deep Learning* (2023)

$$x_n \longrightarrow \text{softmax}[\text{pessoa, local, organização, nenhuma}]$$

Cada uma das N posições gera sua própria distribuição de probabilidade e seu próprio termo de perda — diferente da classificação de sequência, que usa só uma posição.

Encoder: representação, não geração

Um encoder como o BERT constrói uma **representação bidirecional** do texto, pré-treinada por auto-supervisão (prever palavras escondidas) e depois especializada por fine-tuning para tarefas específicas — classificação e NER.

Na Aula 4: o **decoder** (GPT3) — que não constrói uma representação para reaproveitar, mas gera texto token a token — e o **encoder-decoder**, que combina os dois para tarefas como tradução.